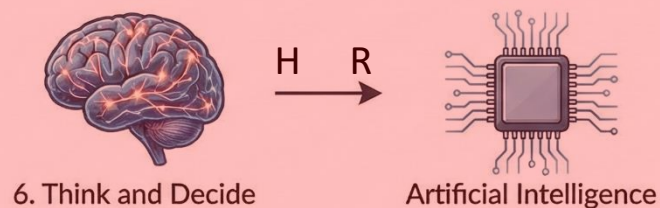
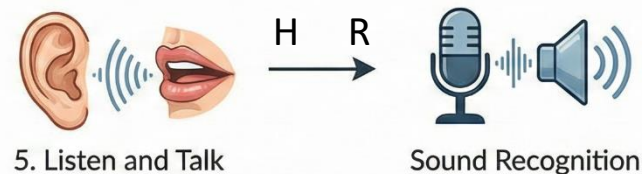
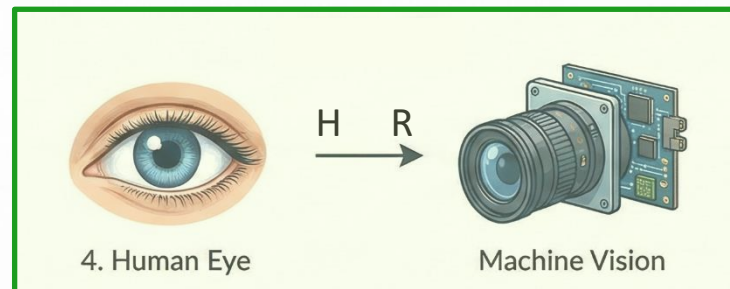
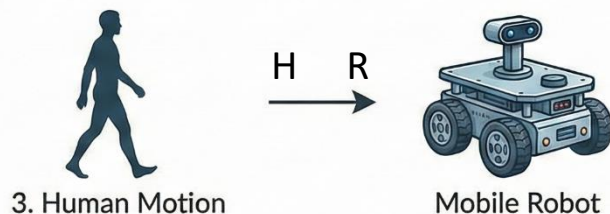
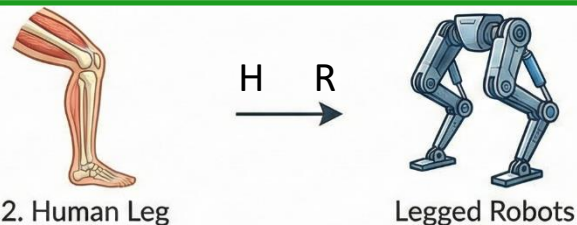
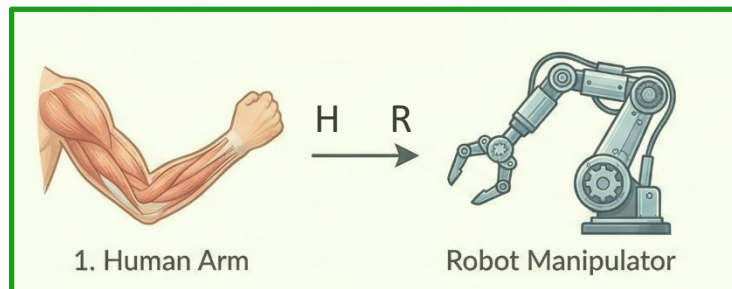


ARAS Robotics & Neura Computer Vision

Chapter 6: Convolutional neural networks

convolution/padding/stride; receptive fields; pooling; transfer learning.



Object detection goes beyond basic image labeling by simultaneously identifying and precisely locating multiple items.

- **Classification vs. Detection**

- **Classification:** Outputs a single label and probability per image.
- **Detection:** Locates multiple objects within complex, real-world scenes.

- **Bounding Box Mechanics**

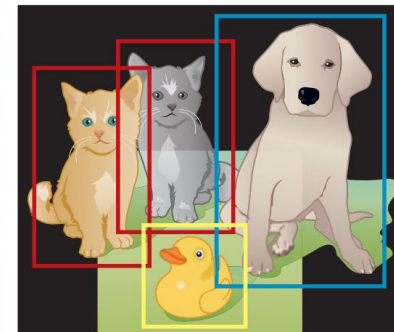
- **Output Format:** Defined by 4 coordinates: center (x,y), width (w), height (h).
- **Dual Challenge:** Performs spatial localization (where) and classification (what) at the same time.

Image classification



Cat

Object detection
(classification and localization)



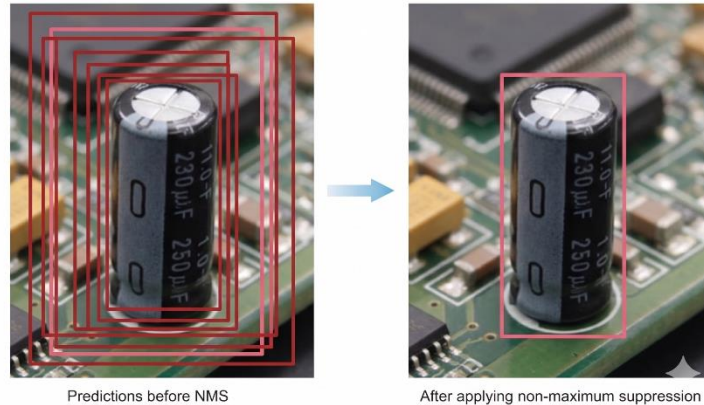
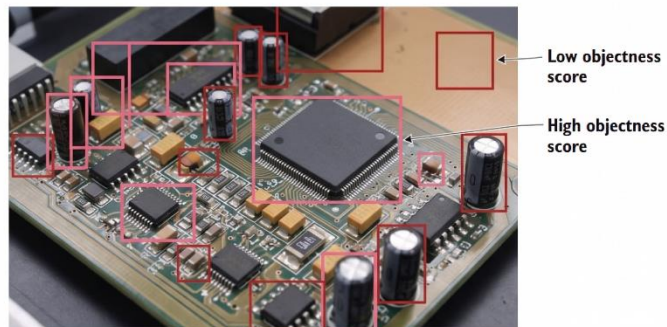
Cat, Cat, Duck, Dog

Image classification	Object detection
The goal is to predict the type or class of an object in an image. ■ Input: an image with a single object ■ Output: a class label (cat, dog, etc.) ■ Example output: class probability (for example, 84% cat)	The goal is to predict the location of objects in an image via bounding boxes and the classes of the located objects. ■ Input: an image with one or more objects ■ Output: one or more bounding boxes (defined by coordinates) and a class label for each bounding box ■ Example output for an image with two objects: – box1 coordinates (x, y, w, h) and class probability – box2 coordinates and class probability Note that the image coordinates (x, y, w, h) are as follows: (x and y) are the coordinates of the bounding-box center point, and (w and h) are the width and height of the box.

The Object Detection Pipeline

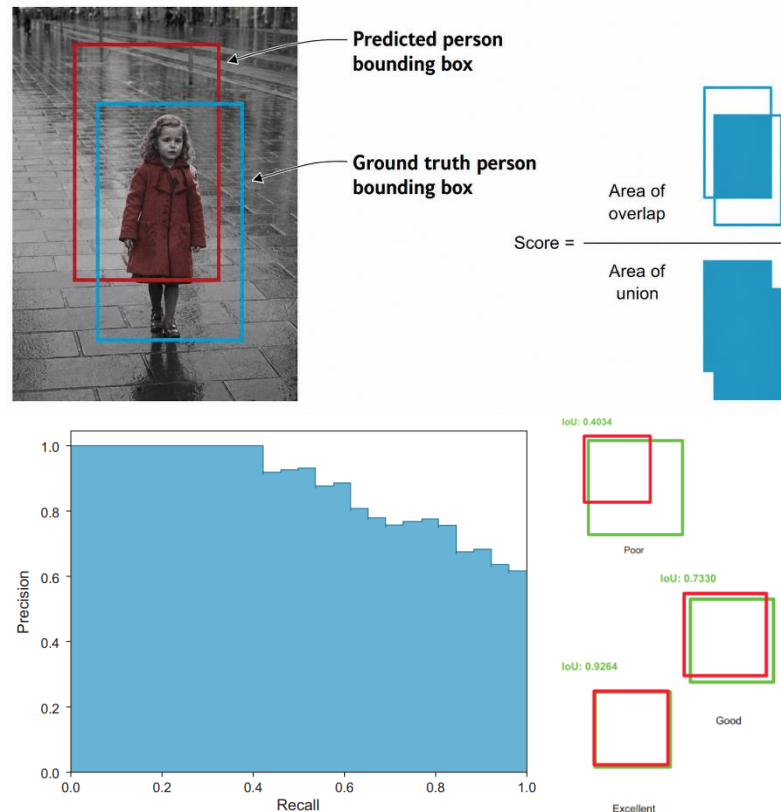
Modern detection systems follow a structured framework to propose, classify, and filter object bounding boxes.

- **1. Region Proposals (RoI)**
 - Identifies areas with high "Objectness Scores"
 - Uses thresholds to filter out empty backgrounds
- **2. Network Predictions**
 - Classifies the object's category (using Softmax)
 - Outputs refined bounding box coordinates (x, y, w, h)
- **3. Non-Maximum Suppression (NMS)**
 - Selects the highest-confidence prediction per object
 - Deletes redundant overlapping boxes using IoU thresholds



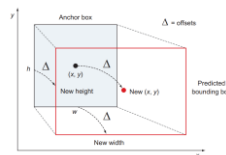
Assessing detection models requires balancing processing speed (FPS) with robust spatial accuracy (mAP).

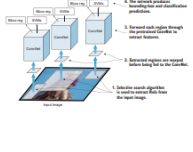
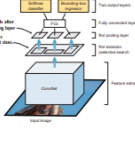
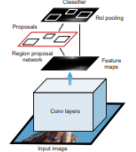
- **Spatial Accuracy: IoU (Intersection over Union)**
 - Measures overlap between predicted and ground truth boxes
 - Dictates True Positives (typically $\text{IoU} > 0.5$)
- **The Precision-Recall Trade-off**
 - **Precision:** Trustworthiness of predictions (minimizes false alarms)
 - **Recall:** Total detection capability (minimizes missed objects)
- **The Ultimate Standard: mAP**
 - **AP:** Calculated as the area under the Precision-Recall curve
 - **Mean Average Precision (mAP):** Averages AP scores across all classes



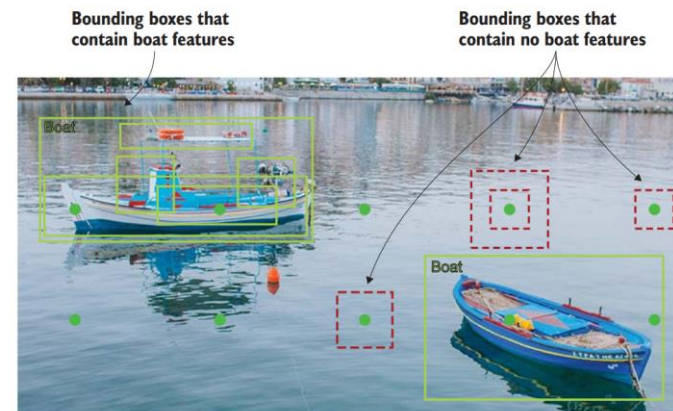
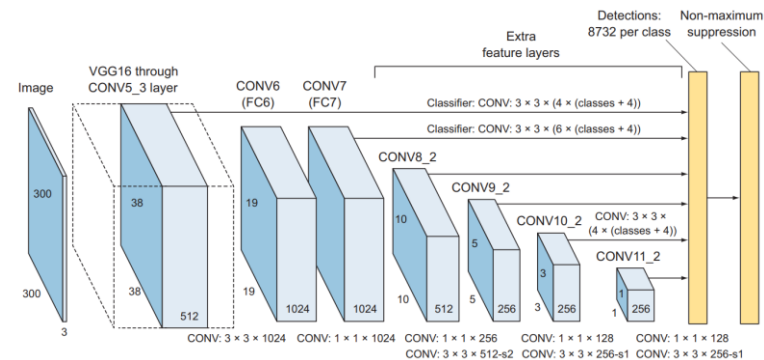
R-CNN models evolved from multi-stage, slow pipelines into unified, GPU-accelerated architectures.

- R-CNN (The Pioneer):**
 - Used Selective Search (CPU) to propose 2000 regions.
 - Independent training stages; slow (50s/image).
- Fast R-CNN (The Reformer):**
 - Shared CNN computation via RoI Pooling.
 - Unified training; 25x faster (2s/image).
- Faster R-CNN (The King):**
 - RPN: Replaced Selective Search with a CNN-based region proposal network.
 - Anchor Boxes: 9 multi-scale templates per pixel for robust detection.
 - Multi-Task Loss: Balanced classification and regression via a λ weighting factor.
 - Performance: Real-time speed (0.2s/image) with high accuracy.



	R-CNN	Fast R-CNN	Faster R-CNN
			
mAP on the PASCAL Visual Object Classes Challenge 2007	66.0%	66.9%	66.9%
Features	- Applies selective search to extract RoIs (~2,000) from each image. - A ConvNet is used to extract features from each of the ~2,000 regions extracted. - Uses classification and bounding box predictions.	Each image is passed only once to the CNN, and feature maps are extracted. - A ConvNet is used to extract feature maps from the input image. - Selective search is used on these maps to generate predictions. This way, we run only one ConvNet over the entire image instead of ~2,000 ConvNets over 2000 overlapping regions.	Replaces the selective search method with a region proposal network, which makes the algorithm much faster. An end-to-end DL network.
Limitations	High computation time, as each region is passed to the CNN separately. Also, uses three different models for making predictions.	Selective search is slow and, hence, computation time is still high.	Object proposal takes time, and as there are different systems working one after the other, the performance of systems depends on how the previous system performed.
Test time per image	50 seconds	2 seconds	0.2 seconds
Speed-up from R-CNN	1x	25x	250x

- **Single-Stage Efficiency**
 - Bypasses RPN to achieve high-speed tracking (59 FPS)
 - Evaluates 8,732 anchor boxes instantly using NMS
- **Multi-Scale Feature Layers**
 - Early high-resolution layers detect tiny objects
 - Deep low-resolution layers capture large objects
- **Dilated Convolutions**
 - Expands the filter's receptive field via spaced pixels
 - Increases spatial context with zero added computational cost



YOLO processes the entire image once, dividing it into a grid where specific cells are solely responsible for local object detection.

The Grid Architecture

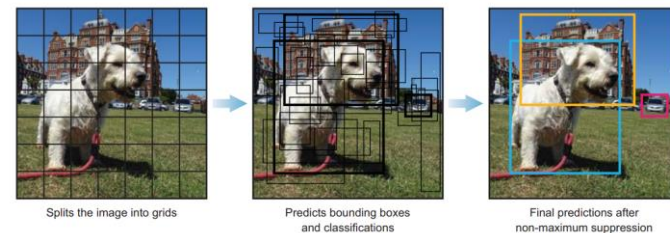
- Divides the image into an $S * S$ grid.
- If an object's center falls in a cell, that cell predicts its bounding box.

Key YOLOv3 Innovations

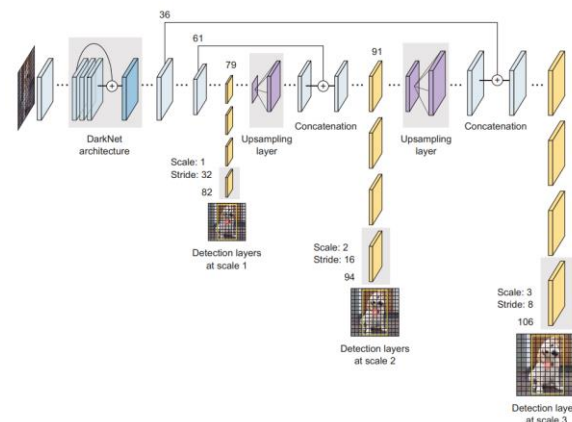
- DarkNet-53 Backbone: Uses residual blocks for deeper, stable feature extraction.
- Multi-Label Classification: Replaces Softmax with Sigmoid for non-exclusive class predictions.

Multi-Scale Predictions

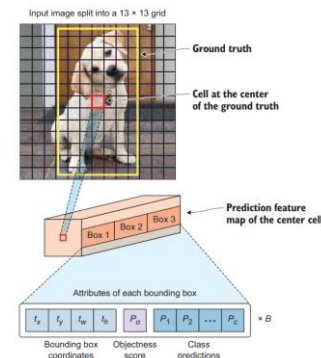
- Detects objects across three scales (e.g., 13x13, 26x26, 52x52). Dramatically improves the detection of small, previously missed objects.



YOLO splits the image into grids, predicts objects for each grid, and then uses NMS to finalize predictions.



YOLOv3 network architecture. (Inspired by the diagram in Ayosh Kuthuria's post "What's new in YOLO v3?" *medium*, 2018, <http://mng.bz/IGN2>.)



Example of a YOLOv3 workflow when applying a 13×13 grid to the input image. The input image is split into 169 cells. Each cell predicts B number of bounding boxes and their objectness score along with their class predictions. In this example, we show the cell at the center of the ground-truth making predictions for 3 boxes ($B = 3$). Each prediction has the following attributes: bounding box coordinates, objectness score, and class predictions.